# Ders 22 Çalışma Özeti

## Ders 22 Çalışma Özeti

### Genel Konular

- Dersin tarihi 9 Haziran 2021 Çarşamba. Ders Zoom üzerinden yapıldı.
  - Asıl ders yaklaşık 00:05'te başlar.
  - Dersin son konu anlatımıdır. Müfredat geçen hafta bitti; bu derste ek konular işlendi.
- Üretici-tüketici problemi (producer-consumer) uygulaması
  - Kapasitesi sınırlı bir `LinkedList` tampon (buffer) olarak kullanıldı.
  - Üretici thread sona ekler, tüketici thread baştan alır (`removeFirst`).
  - `synchronized (this)`, `wait()` ve `notify()` ile senkronizasyon yapıldı.
  - Derste kod tam olarak düzeltilemedi. Hoca doğru çözümü sonra paylaşacağını söyledi. Doğru sürüm "Detaylı Açıklamalar" kısmında.
- Java Swing ile grafik arayüz (GUI)
  - Java GUI kütüphaneleri: AWT (eski), Swing, JavaFX.
  - `JFrame`, `JButton`, `JTextField`, `setBounds`, `setLayout(null)`, `setSize`, `setVisible`, `setDefaultCloseOperation`.
  - Olay yönetimi (event handling): `addActionListener`, `ActionListener`, `actionPerformed`, `ActionEvent`, `getSource()`.
  - Olay dinleyicisini yazmanın üç yolu: sınıfın kendisi (`this`), ayrı (dış) sınıf, anonim iç sınıf.
  - Buton dizisi (`JButton[]`) ile çok sayıda butonu tek metotta yönetme.
  - Python'da PyQt ile `connect` kullanımına kısa bakış.
- Katmanlı mimari (layered / n-tier architecture)
  - Veri tabanı katmanı, iş mantığı katmanı (business logic layer), façade katmanı, kullanıcı arayüzü.
- MVC (Model-View-Controller)
  - ASP.NET MVC örneği: `StudentController`, `Index` view'u, `Student` modeli, Razor sözdizimi.
- Öğrenci sunumu (Ömer): MVP, MVVM ve Clean Architecture
  - MVP: Presenter, Controller'ın görevlerini daraltır.
  - MVVM: View ile ViewModel arasında veri bağlama (data binding).
  - Android üzerinde MVC, MVP ve MVVM ile yazılmış giriş (login) örneği.
- Kapanış: yazılım tasarımı, tasarım kalıpları (design patterns) ve ORM araçları (Entity Framework, Hibernate) üzerine tavsiyeler.

### Hocanın Özellikle Vurguladığı Kısımlar

- Paylaşılan kaynağa (buradaki liste) erişim senkron olmalı.
  - Biri erişirken diğeri erişmemeli. Yoksa üretici daha eklemeyi bitirmeden tüketici araya girebilir.
- Thread işleri algoritmik düşünmenin de ötesinde karışıktır.
  - Üretici-tüketici problemi meşhur ve zor bir problemdir. Üzerinde siz de uğraşın.
- GUI kodu, veri tabanı kodu ve iş mantığı aynı sınıfta olmamalı.
  - Hepsi bir aradayken kod çok kötü görünür. Katmanlara bölün.
- Çok sayıda buton varsa (ör. 40.000 koltuklu stadyum) her biri için ayrı metot yazmayın.
  - Ortak bir metot yazın ve hangi butona basıldığını `getSource()` ile bulun.
- Bu dersten sonra tasarım kalıplarını öğrenin.
  - Piyasa ve yazılım mühendisliği bu yönde ilerliyor. MVC bunlardan biri.

- SQL cümlesi yazmak çok zor değil; ORM araçları bunu sizin yerinize yapıyor.
  - Asıl önemli olan veri tabanının ve nesne yönelimli tasarımın iyi yapılması, kod kusurlarından (code smell) uzak durmak.
- Tasarım kalıpları için Türkçe kaynaklar da artık iyi.
  - Hoca bu konuda kitabı olan yazarlar andı; isimler altyazıda net anlaşılmıyor ("Aykut Taş, Telen" diye geçiyor).

## Kısa Tekrar Notları

- `wait()` ve `notify()` yalnızca `synchronized` blok veya metot içinden çağrılabilir. Dışarıda çağrılırsa `IllegalMonitorStateException` oluşur.
- `wait()` her zaman bir `while` döngüsü içinde koşul kontrolüyle kullanılır: `while (list.size() == capacity) wait();`
- `while (...);` sonuna konan noktalı virgül boş döngü oluşturur. Altındaki satır döngünün parçası olmaz.
- `JFrame.EXIT_ON_CLOSE` değeri 3'tür. Pencere kapanınca program da sonlanır. Varsayılan `HIDE_ON_CLOSE` (1) yalnızca pencereyi gizler.
- `setLayout(null)` ile bileşenler `setBounds(x, y, genişlik, yükseklik)` koordinatlarına yerleşir.
- `JTextField` için `getText()` ve `setText(...)` kullanılır.
- `ActionListener` arayüzünün tek metodu `actionPerformed(ActionEvent e)`'dir.
- `e.getSource()` olayı tetikleyen bileşeni döndürür.
- Java'da metot içinde `static` yerel değişken tanımlanamaz. Sayaç gerekiyorsa sınıf alanı (field) yapılmalı.
- MVC: Model (veri), View (görüntü), Controller (ikisini bağlayan kontrol).
- MVP: Presenter, View ile Model arasında köprü kurar. Test edilebilirliği artırır.
- MVVM: View, ViewModel'ı veri bağlama ile dinler. View pasiftir.

## Detaylı Açıklamalar

### Üretici-tüketici problemi

Senaryo: - Kapasitesi sınırlı bir tampon var. - Tampon doluysa üretici bekler. Boşsa tüketici bekler. - Üretici bir eleman ekleyince tüketiciyi uyandırır (`notify`). Tüketici bir eleman alınca üreticiyi uyandırır. - Main thread, `t1.join()` ve `t2.join()` ile iki thread'in bitmesini bekler.

Derste karşılaşılan hatalar: - İlk sürümde `wait()` ve `notify()` `synchronized` dışında çağrıldı. Program "monitör" hatası verdi (`IllegalMonitorStateException`). - `while (list.size() == 0);` satırının sonunda noktalı virgül vardı. Hoca bunu fark edip sildi. - Asıl sorun döngü koşuluydu. İki thread de yalnızca üretim sayacına (`uretimSayisi`) bakıyordu. - Tüketici `while (uretimSayisi < MAX)` ile dönerse: üretici son elemanı üretince sayaç `MAX` olur. Tüketici listede eleman kalsa bile döngüden çıkar. Son elemanlar tüketilmez. - `<=` yapılırsa: tüketici son elemanı da aldıktan sonra tekrar döngüye girer, liste boş olduğu için `wait()`'e geçer. Üretici bitmiş olduğu için onu kimse uyandırmaz. Program takılır. - Derste bir öğrencinin önerdiği gibi tüketilen eleman sayısı için ayrı bir sayaç gerekir.

Doğru çalışan sürüm (Java 25 ile çalıştırıldı; 10 eleman üretildi, 10'u da tüketildi, ardından "program bitti" yazdı):

```
import java.util.LinkedList;

public class PC {
    private final LinkedList<Integer> list = new LinkedList<>();
    private final int capacity = 5;
    private final int max;
    private int uretimSayisi = 0;
    private int tuketimSayisi = 0;

    public PC(int max) { this.max = max; }

    public void produce() throws InterruptedException {
```

```
        int value = 0;
        while (true) {
            synchronized (this) {
                if (uretimSayisi == max) return;
                while (list.size() == capacity)
                    wait();
                System.out.println("Producer produced " + value);
                list.add(value++);
                uretimSayisi++;
                notify();
            }
            Thread.sleep(50);
        }
    }

    public void consume() throws InterruptedException {
        while (true) {
            synchronized (this) {
                if (tuketimSayisi == max) return;
                while (list.isEmpty())
                    wait();
                int val = list.removeFirst();
                tuketimSayisi++;
                System.out.println("Consumer consumed " + val);
                notify();
            }
            Thread.sleep(80);
        }
    }

    public static void main(String[] args) throws InterruptedException {
        PC pc = new PC(10);
        Thread t1 = new Thread(() -> { try { pc.produce(); } catch (InterruptedException e) {
        Thread t2 = new Thread(() -> { try { pc.consume(); } catch (InterruptedException e) {
        t1.start();
        t2.start();
        t1.join();
        t2.join();
        System.out.println("program bitti");
    }
}
```

- Derste thread'ler anonim `Runnable` sınıfıyla oluşturuldu. Yukarıdaki lambda ifadesi aynı işi kısa yoldan yapar.
- `Thread.sleep` çağrısı `synchronized` bloğun dışında. Böylece uyuyan thread kilidi tutmaz, diğer thread çalışabilir.

### Swing ile basit pencere

Derste yazılan `Pencerem` sınıfının toparlanmış hali (kod derlendi; grafik ortam gerektirdiği için ekranda çalıştırılmadı):

```
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.*;

public class Pencerem {
    private JButton b;
    private JTextField tf1, tf2;
    private static int value = 0;
```

```
    public Pencerem() {
        JFrame f = new JFrame();
        b = new JButton("Tıkla");
        b.setBounds(50, 100, 95, 30);
        f.add(b);
        b.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                button_click(e);
            }
        });
        tf1 = new JTextField("");
        tf1.setBounds(50, 50, 95, 30);
        f.add(tf1);
        tf2 = new JTextField("Merhaba");
        tf2.setBounds(150, 50, 95, 30);
        f.add(tf2);
        f.setSize(300, 300);
        f.setLayout(null);
        f.setVisible(true);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }

    private void button_click(ActionEvent e) {
        value++;
        tf1.setText(tf2.getText() + ", butona " + value + " kez tıklandı");
    }

    public static void main(String[] args) {
        new Pencerem();
    }
}
```

- `setLayout(null)` yazılmadan önce bileşenler varsayılan yerleşim düzenine göre yerleşti ve buton görünmedi.
- `setDefaultCloseOperation` yazılmadan önce pencere kapatılınca program arkada çalışmaya devam etti.
- Butona her tıklamada metin "butona 1 kez tıklandı", "butona 2 kez tıklandı" diye değişti.

### Birden fazla buton ve getSource

```
public void actionPerformed(ActionEvent e) {
    if (e.getSource() == b1)
        System.out.println("Birinci butona basıldı");
    else if (e.getSource() == b2)
        System.out.println("İkinci butona basıldı");
}
```

- Sınıf `ActionListener`'ı uygular ve butonlara `addActionListener(this)` ile eklenirse bütün butonların olayları aynı metoda gelir. Hangi butona basıldığı `getSource()` ile ayrılır.
- Çok sayıda buton için dizi kullanılır: `JButton[] B = new JButton[100];` Sonra her eleman ayrıca `new JButton(...)` ile oluşturulmalıdır.
- Farklı bileşenlerin farklı dinleyicileri vardır: `ActionListener` (buton, menü), `ItemListener` (onay kutusu, seçim kutusu), `MouseListener`, `MouseMotionListener`, `MouseWheelListener`, `KeyListener`.

### Katmanlı mimari ve MVC

- Gerçek uygulamada üç iş vardır: veri tabanı işlemleri, iş mantığı (algoritmalar, kontroller) ve kullanıcı arayüzü.
- Katmanlı mimaride her biri ayrı sınıf grubunda yazılır. Arayüz, verinin veri tabanından geldiğini bilmez.

- Façade katmanı: arayüzden gelen ve iş mantığından geçen veriyi veri tabanına ileten "vitrin" katmanı.
- Hocanın benzetmesi: 100 karpuz taşınırken herkes yürümek yerine karpuzu yanındakine atar. Her katman kendi işini yapar.
- MVC'de:
  - Model: veri tabanı tablosunun sınıf karşılığı (ör. `Student` sınıfında `Id`, `Name`, `Age`). İlkel veri takıntısından (primitive obsession) kurtarır.
  - View: kullanıcının gördüğü sayfa. ASP.NET'te HTML ile C# karışık Razor sözdizimiyle yazılır.
  - Controller: View ile Model'i bağlar. Örneğin `StudentController.Index()` öğrenci listesini alıp `Index` view'una gönderir.

**MVP ve MVVM (öğrenci sunumu)**

- MVC'de Controller çok büyüyebilir ("massive view controller" sorunu).
- MVP:
  - Presenter, View ile Model arasında veri dönüşümünü yapar. Örnek: veri tabanındaki doğum tarihini yaşa çevirmek.
  - İş mantığı kontrolleri (ör. boş kitap adını kaydetmemek) Presenter'da yapılır.
  - Arayüzler (interface) ve zayıf referanslar sayesinde bağımlılık döngüsü (dependency cycle) oluşmaz.
  - Avantaj: parçalar ayrı ayrı birim testle test edilebilir. Dezavantaj: bileşen sayısı fazla, küçük uygulamalarda kurulumu zor.
- MVVM:
  - ViewModel görüntü mantığını tamamen kendisi oluşturur.
  - View, ViewModel'ı veri bağlama (data binding) ile dinler. View pasiftir.
  - Reaktif programlamaya dayanır. Mobil geliştirmede (iOS, Android) çok yaygındır.
- Clean Architecture: Robert C. Martin'in (Uncle Bob) yaklaşımı. Dış katmandan iç katmana: UI, controller, use case, entity. İç katman dış katmanı bilmez. Swift'teki karşılığı VIPER.

**Ders sonrası notlar**

- **`static` yerel değişken:** Hoca `button_click` içinde `static int value = 1;` yazdı ve "illegal start of expression" hatası aldı. Java'da metot içinde `static` değişken tanımlanamaz. Tıklama sayısını tutmak için değişken sınıf alanı (`private static int value;` veya `private int value;`) olmalıdır.
- **İlk `tf2` hatası:** Kopyalanan kodda `tf2` oluşturulduktan sonra yine `tf1.setBounds` ve `f.add(tf1)` yazılmıştı. Bu yüzden ikinci kutu görünmedi. Hoca sonra bunları `tf2` olarak düzeltti.
- **`setDefaultCloseOperation(3)`:** Sabit sayı yerine `JFrame.EXIT_ON_CLOSE` yazmak okunabilirlik için daha doğrudur. Değerler: `DO_NOTHING_ON_CLOSE` = 0, `HIDE_ON_CLOSE` = 1, `DISPOSE_ON_CLOSE` = 2, `EXIT_ON_CLOSE` = 3.
- **Swing ve thread:** Swing bileşenleri thread-safe değildir. Pencereyi oluşturan kod normalde `SwingUtilities.invokeLater(() -> new Pencerem());` ile olay thread'inde (Event Dispatch Thread) çalıştırılmalıdır. Derste bu konuya girilmedi.
- **Buton dizisi:** Hoca önce C# sözdizimiyle karıştırdı. Java'da `JButton[] B = new JButton[100];` yalnızca 100 boş referans oluşturur. Her buton `B[i] = new JButton("...");` ile ayrıca oluşturulmalıdır.
- **`JFrame` için olay dinleyicisi:** Öğrencinin sorusuna hoca "muhtemelen component veya container listener" dedi. Pencere olayları (kapatma, simge durumuna küçültme) için `WindowListener` kullanılır. `JFrame`'de `addActionListener` yoktur.
- **MVC'nin tarihi:** Sunumda MVC'nin "80'lerin sonunda" ortaya çıktığı söylendi. MVC, 1970'lerin sonunda Trygve Reenskaug tarafından Smalltalk için tanımlandı.
- **MVP'nin kökeni:** Sunumda "Intelligent Inc." diye geçen şirket Taligent'tır (Apple, IBM ve HP ortaklığı). MVP 1990'ların başında burada ortaya çıktı.

- **Façade:** Kelime Fransızcadan gelir ve “cephe, ön yüz” demektir. Yazılımda Façade aynı zamanda bir tasarım kalıbının adıdır: karmaşık bir alt sistemi tek ve basit bir arayüzün arkasına saklar.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.